AULA 4 · TURMA LIVE

O seu TP1, conferido e entregue — e as interfaces do seu projeto

Etapa 3 · segunda, 09/11/2026 · Zoom · 1h30 · Prof. Dácio Moreira de Souza

Hoje o seu projeto fecha uma entrega — e para de falar a língua dos outros.

São duas metades. Na primeira, o TP1: como conferir que o repositório mostra o que o relatório diz, e como entregar sem surpresa. Na segunda, o começo do TP2: a mensagem do seu sistema deixa de ser um número solto e passa a ter o vocabulário do seu domínio.

Cenas dos últimos episódios: o que a Aula 3 deixou

- A imagem entrou no grafo: sensor_msgs/Image em /camera/image_raw, com QoS de sensor
- A cor virou contagem: HSV, máscara, contornos — e um Int32 em /vision/contagem
- O /vision/status respondeu a uma pergunta, no tipo pronto std_srvs/Trigger
- A cor trocou com o sistema no ar: ros2 param set, sem recompilar nada
- E a contagem caiu de 2 para 1 quando os objetos se encostaram: oclusão
- Na semana sem aula: as duas classes do SEU domínio, o YAML e as evidências

Cenas: as peças antigas que voltam hoje

- Pacote ament_python (Aula 2): hoje ele ganha um vizinho, só de definições
- Serviço é pergunta com resposta (Aula 2): hoje a resposta ganha campos com tipo
- Todo tópico tem um TIPO (Aula 2): hoje o tipo passa a ser seu
- source install/setup.bash em cada terminal (Aula 2): hoje ele decide se a interface existe
- Parâmetro fora do código (Aula 3): hoje o rótulo da classe entra por parâmetro
- Oclusão (Aula 3): hoje ela aparece também como confiança caindo

O que você sai sabendo hoje

- Conferir, com quatro comandos, se o seu TP1 mostra o que diz
- Escrever um relatório que responde perguntas, em vez de narrar o mês
- Entregar sem surpresa: main, entrega-tp1, a tag, o Moodle e a janela anônima
- Dizer quando um tipo pronto basta — e quando o projeto precisa do próprio
- Criar, compilar e usar um .msg e um .srv num pacote de interfaces

E o que cada uma vale no SEU projeto

- Conferir: você descobre hoje, e não no dia da entrega, o que falta mostrar
- Relatório: é nele que o seu domínio aparece — o código do exemplo é igual para todos
- Entrega: o que o avaliador não consegue abrir não existe
- Tipo próprio: a mensagem passa a dizer "2 tampas azuis", e não "2"
- Pacote de interfaces: é o contrato que o seu detector treinado vai reaproveitar
- As datas e os gates estão na página e em "Gates de cada TP" — consulte, não decore

Como a aula funciona hoje

- Eu demonstro no meu repositório, o pb-live-dacioms; vocês observam e anotam o que procurar
- Primeira metade: o TP1, pelos três casos que mais travam uma entrega
- Segunda metade: interfaces próprias, com os arquivos abertos e os comandos na tela
- O que é para rodar no SEU ambiente está na página: a tarefa da semana, em ordem
- Se o tempo apertar, o bloco 3 cede: ele está inteiro na página e no guia de estudo

Antes de começar: uma letra no chat

- A — ainda não fechei o projeto, ou tenho uma classe só
- B — a imagem ainda não circula no meu repositório, ou não medi a taxa
- C — a segmentação ou o /vision/status ainda são os do exemplo
- D — o código está de pé; falta o relatório e a entrega
- E — estou sem ambiente ou sem repositório (escreva o sintoma)
- Não é chamada nem nota: é para eu escolher por onde a primeira metade começa

BLOCO 1

O seu TP1, conferido e entregue

O que o TP1 mostra, em quatro capacidades

- O ambiente funciona, e você sabe dizer quem publica, quem assina e quem serve
- A imagem circula no grafo, e a taxa está medida e explicada
- Um nó segmenta o SEU objeto por cor, conta, e responde ao /vision/status
- O OpenCV básico aparece: cores e contornos — e faces/features como demonstração
- Tudo isso sobre um projeto declarado: domínio, usuário, duas classes e plano B
- O detalhe de cada item está na página do TP1: é referência, para consultar

Terminal A: o seu sistema no ar

```
export PB_USER=dacioms
export PB_DIR="$HOME/pb-live-$PB_USER" && export PB_WS="$PB_DIR/ros2_ws"
source /opt/ros/humble/setup.bash
source "$PB_WS/install/setup.bash"
ros2 launch aula03_visao visao.launch.py

# Publicando /camera/image_raw · fonte=sintetico · 640x480 @15fps · cv_bridge=sim
# Segmentador no ar: /camera/image_raw -> /vision/segmented + /vision/contagem
#                    · serviço /vision/status
```

Terminal novo começa SEMPRE pelas variáveis e pelas duas linhas de source. No seu projeto o pacote tem o SEU nome (percepcao_estoque, visao_pomar...) — o resto é igual. O launch ocupa este terminal: as perguntas vão para outro.

Terminal B: quatro perguntas ao sistema

```
# Terminal B, novo: as mesmas variáveis e os dois source do Terminal A
ros2 node list                                  # /publicador_camera  /segmentador_hsv
ros2 topic echo /vision/contagem --once         # data: 2
ros2 service call /vision/status std_srvs/srv/Trigger "{}"
#   success=True, message='2 objeto(s) no ultimo frame (239 frames processados)'
ros2 topic hz /vision/contagem                  # average rate: 15.0   (Ctrl+C para sair)
```

Quem está no ar, o que ele vê, o que ele responde, e a que taxa. São as quatro respostas que o relatório cita — e cada uma pode virar um arquivo no repositório (é o caso 3).

A taxa: 5 Hz antes, 15 Hz depois

```
ros2 topic hz /vision/contagem
#   average rate: 5.676     min: 0.000s  max: 0.867s     <- quadros se perdendo

# uma vez por máquina: o perfil do Fast DDS (o arquivo está na página, pronto para colar)
export FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/.ros/fastdds-imagem.xml"
# ...e o launch do Terminal A de novo, com a variável já definida

ros2 topic hz /vision/contagem
#   average rate: 15.001    min: 0.065s  max: 0.069s
```

A câmera não tem culpa. Uma imagem de 640x480 pesa 0,92 MB e não cabe no canal de memória compartilhada que o ROS 2 usa, por padrão, entre processos da mesma máquina: quadros caem, sem mensagem de erro. Taxa baixa medida e explicada vale; taxa baixa escondida, não.

Caso 1 — o projeto diz o que a câmera procura

PROJETO.md

```
<!-- PB:PROJETO -->
**Domínio:** colheita em pomar de tomate, setor 3
**Usuário:** o encarregado da colheita, que decide quando colher cada fileira
**Classes percebidas no TP1:** classe A = tomate maduro · classe B = tomate verde
**Fonte de imagem no TP1:** cena sintética com as duas classes; a webcam entra depois
<!-- PB:TRILHA -->
**Trilha:** S (simulada), com a webcam como único elemento real mais adiante
**Plano B:** se a câmera falhar na semana da entrega, cena sintética e um vídeo gravado
```

Uma classe só esconde o problema: com uma classe, qualquer limiar que detecte alguma coisa parece funcionar. No seu projeto, é o DOMÍNIO que decide as duas — tampa azul e tampa verde, capacete e colete. Sem câmera, a cena sintética vale, dita no relatório.

Caso 2 — o status que fala a língua do domínio

aula03_visao/segmentador_hsv.py

```
        self.declare_parameter('dominio', 'pomar setor 3')     # de quem e o problema
        self.declare_parameter('rotulo', 'tomate maduro')      # o que a camera procura
        ...
    def on_status(self, req, resp):
        resp.success = self.frames > 0
        dominio = self.get_parameter('dominio').value
        rotulo = self.get_parameter('rotulo').value
        resp.message = (f'{dominio}: {self.ultima_contagem} {rotulo} no ultimo frame '
                        f'({self.frames} frames processados)')
        return resp
```

Quatro linhas a mais do que o exemplo. O domínio e o rótulo entram como PARÂMETROS: a mesma função serve ao pomar e ao estoque, sem editar o código. No seu projeto, a frase é a que o usuário do sistema gostaria de ler.

Caso 2, a prova: a mesma pergunta, outra resposta

```
ros2 service call /vision/status std_srvs/srv/Trigger "{}"
#   antes:   message='2 objeto(s) no ultimo frame (239 frames processados)'
#   depois:  message='pomar setor 3: 2 tomate maduro no ultimo frame (99 frames processados)'

ros2 param set /segmentador_hsv dominio "estoque doca 2"
ros2 param set /segmentador_hsv rotulo "caixa vermelha"
ros2 service call /vision/status std_srvs/srv/Trigger "{}"
#   message='estoque doca 2: 2 caixa vermelha no ultimo frame (232 frames processados)'
```

O serviço é o mesmo, o tipo é o mesmo; o que mudou foi o vocabulário. E repare no incômodo: os números continuam DENTRO de uma frase. Quem chama precisa ler texto para achar o "2". É esse incômodo que a segunda metade da aula resolve.

Caso 3 — o terminal fecha, o repositório fica

```
cd "$PB_DIR" && mkdir -p docs/evidencias/tp1
ros2 service call /vision/status std_srvs/srv/Trigger "{}" > docs/evidencias/tp1/status.txt
timeout 10 ros2 topic hz /vision/contagem > docs/evidencias/tp1/topic-hz.txt
ls docs/evidencias/tp1/                          # status.txt  topic-hz.txt
git add -A
git commit -m "TP1: evidencias do status e da taxa"
git push origin dev
```

O sinal > grava a resposta num arquivo; o timeout encerra a medição sozinho depois de 10 s. Evidência que ficou só no terminal não existe para quem avalia. No seu projeto: um arquivo por afirmação do relatório — o que você diz, o repositório mostra.

A oclusão: observar, e não consertar

```
ros2 topic echo /vision/contagem
#   data: 2
#   ---
#   data: 1          <- os dois objetos se encostaram
#   ---
#   data: 2
```

Ninguém saiu de cena: o sistema perdeu a capacidade de distinguir. Um print desse momento e três linhas no relatório — o que aconteceu, por quê, e o que isso significa no SEU domínio. Dois tomates encostados virando um: a contagem da colheita erra para menos.

O relatório responde a cinco perguntas

- Que problema, de quem — e por que este, e não outro?
- O que você escolheu, e o que DESCARTOU: fonte de imagem, faixas de cor, área mínima, QoS
- O que os números dizem: a taxa medida, a contagem, a oclusão
- O que o sistema NÃO faz — dito por você, antes de ser dito por outro
- O que a IA gerou, o que você revisou, e o que você sabe explicar
- O modelo tem oito seções; estas cinco perguntas dizem o que escrever nelas

A entrega, em quatro linhas

```
git checkout main && git merge dev && git push origin main                          # 1) main recebe
git checkout entrega-tp1 && git merge --ff-only main && git push origin entrega-tp1  # 2) entrega = main
git tag -a tp1 -m "Entrega do TP1" && git push origin tp1                            # 3) a marca
git checkout dev                                                                     # 4) de volta

git archive --format=zip --prefix=tp1/ -o ~/tp1-codigo.zip tp1                       # o ZIP do código
```

A entrega OFICIAL é no Moodle: o PDF do relatório, o ZIP do código e os links. O GitHub é complementar e obrigatório, e a tag marca o commit que será corrigido. A branch entrega-tp1 já existe desde o init-branches.sh: é checkout, e não checkout -b.

Antes de enviar: o avaliador consegue abrir?

- O ZIP tem o código, e não tem build/, install/, log/ nem .git/
- O PDF do relatório mostra as imagens — referência quebrada vira espaço em branco
- O vídeo tem até 5 minutos e é público ou não listado — nunca privado
- Uma janela anônima, e um clique em cada link do relatório
- O repositório é o pb-live-<usuario> da organização, com o convite aceito
- O passo a passo, com os comandos, é o "Checklist de entrega do TP1", no site

Não vai dar tempo de tudo? Entregue o que funciona e declare o que não funciona.

Reduza o escopo sem abandonar o item: cena menor, fonte sintética, uma classe a menos. Escreva a limitação no relatório, com as suas palavras. E entregue no prazo. Limitação escrita vale mais do que falha descoberta — e repositório vazio não tem degrau.

BLOCO 2

Quando a mensagem pronta acaba

Dois. Dois o quê?

```
ros2 topic echo /vision/contagem --once
#   data: 2

ros2 interface show std_msgs/msg/Int32
#   # It is recommended to create your own semantically meaningful message.
#   int32 data
```

Dois de que classe? Com que certeza? Onde na imagem? De que instante? O segmentador conhece a posição, a área e o contorno de cada objeto — e joga tudo fora na hora de publicar, porque o tipo só tem lugar para um inteiro. Não é limite do ROS 2: é escolha de tipo, e o próprio Int32 recomenda, no comentário, criar uma mensagem com significado.

Tipo pronto, tipo próprio — e o atalho que cobra depois

- Tipo pronto (Int32, Trigger, Image): certo enquanto a pergunta é simples. "Quantos?" é Int32
- Tipo próprio: quando três ou mais campos andam sempre juntos
- ...ou quando dois tópicos só fazem sentido lidos ao mesmo tempo
- Ele custa: um pacote a mais, um build a mais, e uma dependência para quem usar o seu nó
- O atalho: JSON dentro de uma String. Funciona — e perde tipo, inspeção e o lado C++
- No seu projeto: o que o seu nó sabe sobre cada objeto, e hoje não conta para ninguém?

Dois pacotes, lado a lado

ros2_ws/src/

```
ros2_ws/src/
|-- pb_interfaces/                  ament_cmake: SO definicoes, nenhum no
|   |-- msg/Deteccao.msg
|   |-- msg/Deteccoes.msg
|   |-- srv/StatusVisao.srv
|   |-- CMakeLists.txt              a lista do que gerar
|   `-- package.xml                 as tres linhas do rosidl
`-- aula06_percepcao/               ament_python: os nos, como desde a Aula 2
    |-- aula06_percepcao/detector.py
    |-- aula06_percepcao/monitor.py
    `-- package.xml                 <depend>pb_interfaces</depend>
```

Gerar interface é gerar CÓDIGO: o ROS 2 lê o .msg e escreve as classes em Python e em C++. Quem faz isso é o rosidl, que roda num pacote ament_cmake — o pacote Python dos seus nós não tem essa maquinaria. É assim que o sensor_msgs chega até você.

O que faz o pacote gerar código

pb_interfaces/package.xml

```
<package format="3">
  <name>pb_interfaces</name>
  <buildtool_depend>ament_cmake</buildtool_depend>

  <buildtool_depend>rosidl_default_generators</buildtool_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>

  <depend>std_msgs</depend>          <!-- Deteccoes.msg usa std_msgs/Header -->
  <export><build_type>ament_cmake</build_type></export>
</package>
```

São estas três linhas. Sem elas o pacote compila, não reclama — e não gera nada: é o pior tipo de erro, o silencioso. No seu projeto, só o <name> muda: <seuprojeto>_interfaces.

Cada arquivo novo entra nesta lista

pb_interfaces/CMakeLists.txt

```
project(pb_interfaces)
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(std_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Deteccao.msg"
  "msg/Deteccoes.msg"
  "srv/StatusVisao.srv"
  DEPENDENCIES std_msgs
)
ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

Criou um .msg e ele "não existe"? Faltou a linha aqui. O std_msgs aparece nos DOIS arquivos — no package.xml e em DEPENDENCIES —, e esquecer um deles é o erro do Header que não resolve. (No exemplo, a lista tem também a ação da próxima aula.)

Uma detecção, e o quadro inteiro

msg/Deteccao.msg · msg/Deteccoes.msg

```
# msg/Deteccao.msg -- uma deteccao individual
string  classe        # rotulo do SEU dominio: "tomate_maduro", "vaga_livre", "peca_ok"
float32 confianca     # 0.0 a 1.0 -- quanto o detector acredita nisso
int32   x             # centro do objeto, em pixels da imagem
int32   y
int32   area          # area em pixels: separa objeto de respingo

# msg/Deteccoes.msg -- o que a percepcao viu em UM quadro
std_msgs/Header header        # stamp: QUANDO   ·   frame_id: em que referencial
Deteccao[]      deteccoes     # array de tamanho variavel de outra mensagem SUA
uint32          total
```

Tipo, nome e comentário — e mensagens se compõem: um campo pode ser outra mensagem sua, e [] faz o array. O Header não é enfeite: sem stamp não se sabe se a detecção é de agora ou de três segundos atrás. No seu projeto: troque o vocabulário, e acrescente o campo que só o seu domínio pede.

A mesma pergunta, com vocabulário próprio

```
ros2 interface show std_srvs/srv/Trigger
#   ---
#   bool success
#   string message

cat pb_interfaces/srv/StatusVisao.srv            # o arquivo, na pasta do exemplo
#   ---
#   bool    ativo
#   string  classe_alvo
#   uint32  total_detectado
#   float32 confianca_media
#   string  mensagem
```

Os três traços separam a pergunta da resposta; aqui a pergunta é vazia nos dois. O Trigger diz "estou vivo". O StatusVisao diz O QUE está sendo visto, em campos com tipo: quem chama não precisa mais procurar um número dentro de uma frase.

Se a resposta demora, não é serviço: é ação.

Quem chama um serviço fica esperando. Para "qual o seu estado?", ótimo. Para "varra a cena inteira e me avise quando terminar", péssimo: tarefa longa pede progresso e cancelamento. É a terceira forma de comunicação do ROS 2, e é a próxima aula — a ação entra neste mesmo pacote de interfaces.

BLOCO 3

Os dois pacotes, de ponta a ponta

Terminal B: copiar e compilar, na ordem

```
cp -r /tmp/PBRoboticos_live_prof_dacio/exemplos/aula06-interfaces/pb_interfaces "$PB_WS/src/"
cp -r /tmp/PBRoboticos_live_prof_dacio/exemplos/aula06-interfaces/aula06_percepcao "$PB_WS/src/"
cd "$PB_WS"
colcon build --packages-select pb_interfaces              # Finished <<< pb_interfaces [10.5s]
source install/setup.bash
colcon build --packages-select aula06_percepcao --symlink-install
source install/setup.bash
```

Primeiro as interfaces, depois o source, depois os nós. Os dez segundos (ou mais) do primeiro build são o CMake gerando código Python e C++ a partir de três arquivos de texto. O source do meio é o que faz o segundo pacote enxergar o primeiro.

Três mensagens, e o que cada uma pede

```
# 1) compilei os nós ANTES das interfaces -- o colcon diz o nome do que falta:
#      Check that the following packages have been built:
#      - pb_interfaces

# 2) terminal sem o source de depois do build -- a lista mostra onde ele procurou:
#      Package 'aula06_percepcao' not found ... searching: ['.../install/aula03_visao', '/opt/ros/humble']

# 3) o nó importa a interface, mas o terminal é antigo (ou falta o <depend>):
#      ModuleNotFoundError: No module named 'pb_interfaces'
```

As três têm a mesma cura em ordem diferente: interfaces compiladas, e source feito DEPOIS, neste terminal. Na dúvida, terminal novo. Anotem as três frases: são elas que vocês vão procurar quando criarem o pacote do próprio projeto.

Terminal B: o teste de fumaça

```
ros2 interface list | grep pb_interfaces
#   pb_interfaces/msg/Deteccao
#   pb_interfaces/msg/Deteccoes
#   pb_interfaces/srv/StatusVisao       (e os dois da próxima aula)

ros2 interface show pb_interfaces/msg/Deteccoes
#   std_msgs/Header header
#   Deteccao[]      deteccoes           (com os campos de Deteccao abertos logo abaixo)
#   uint32          total
```

Se a lista mostra, o ROS 2 enxerga a sua interface — e qualquer nó, em qualquer linguagem, pode usá-la. É a primeira coisa a rodar depois de criar o pacote do seu projeto: se a lista vier vazia, o problema está nas três linhas do package.xml.

Terminal A: rodar, e ler a mensagem nova

```
# Terminal A: Ctrl+C no launch da Aula 3, o source de novo, e então
source "$PB_WS/install/setup.bash"
ros2 launch aula06_percepcao percepcao.launch.py classe:=tomate_maduro

# [detector]: detector no ar - classe='tomate_maduro' - publica pb_interfaces/Deteccoes
#             em /vision/deteccoes - servico /vision/status (pb_interfaces/StatusVisao)
# [monitor_deteccoes]: t=1794265911.67  total=2
#     tomate_maduro(0.98) em (261,333) area=1450 | tomate_maduro(0.99) em (512,216) area=3524
```

Cada linha do monitor é uma mensagem: a classe, a confiança, a posição e a área de cada objeto — tudo o que o Int32 jogava fora. E a classe entrou por parâmetro, no launch: classe:=tomate_maduro. No seu projeto, é aqui que entra o seu rótulo.

Terminal B: a mesma pergunta, a resposta nova

```
ros2 topic echo /vision/deteccoes --once         # o header, o array deteccoes e o total
ros2 service type /vision/status                 # pb_interfaces/srv/StatusVisao
ros2 service call /vision/status pb_interfaces/srv/StatusVisao "{}"
#   ativo=True, classe_alvo='tomate_maduro', total_detectado=2,
#   confianca_media=0.9857, mensagem="vendo 2 'tomate_maduro' na cena"
```

O nome do serviço é o mesmo da primeira metade: /vision/status. O que mudou foi o TIPO — é uma migração, e é o coração da tarefa. total_detectado é um número de verdade: quem chama não lê mais uma frase para achar o "2".

Quem consome a interface

aula06_percepcao/detector.py

```
from pb_interfaces.msg import Deteccao, Deteccoes
from pb_interfaces.srv import StatusVisao
...
        self.pub = self.create_publisher(Deteccoes, '/vision/deteccoes', 10)
        self.srv = self.create_service(StatusVisao, '/vision/status', self.responder_status)
...
            d = Deteccao()
            d.classe = self.classe
            d.confianca = float(min(1.0, area / (cv2.contourArea(cv2.convexHull(c)) + 1e-6)))
...
        resp.total_detectado = len(d)
        resp.confianca_media = float(sum(x.confianca for x in d) / len(d)) if d else 0.0
```

Para o nó, a interface é um import como outro qualquer — e é esse import que falha quando falta o source. A confiança aqui é a SOLIDEZ do contorno (a área dividida pela área do fecho convexo): não é probabilidade, e o código diz isso no comentário.

Os objetos se encostam: a contagem cai, e a confiança também

```
# o que o monitor imprime, no Terminal A, quando os dois objetos se encostam:
#
# total=2  tomate_maduro(0.98) em (276,337) area=1450 | tomate_maduro(0.99) em (511,222) area=3524
# total=1  tomate_maduro(0.89) em (450,290) area=4965
# total=2  tomate_maduro(0.99) em (401,302) area=3524 | tomate_maduro(0.98) em (460,269) area=1450
```

É a oclusão da primeira metade, vista pela mensagem nova: antes só o número mudava; agora o detector mostra que perdeu certeza. Um detector treinado produz esse sinal de forma direta — e usa a MESMA mensagem. Trocar o miolo sem trocar o contrato: é por isso que a interface vem antes do detector.

A tarefa da semana, parte 1: a entrega do TP1, em ordem

- 1. As quatro perguntas no SEU sistema, com as respostas em docs/evidencias/tp1/
- 2. Os três casos de hoje, conferidos no seu repositório: as classes, o status, as evidências
- 3. A oclusão: um print e três linhas
- 4. O relatório, pelas cinco perguntas
- 5. main, entrega-tp1 e a tag — e o Moodle: PDF, ZIP e links, testados em janela anônima
- Os passos, com os comandos e os terminais nomeados, estão na página, em "A tarefa da semana"

Parte 2, depois da entrega: as interfaces do seu projeto

- O que fazer: o seu <projeto>_interfaces, com um .msg e um .srv do seu domínio
- O que usar: o pb_interfaces do exemplo, copiado e renomeado em três lugares
- Como adequar: troque o vocabulário, e acrescente o campo que só o seu domínio pede
- Como saber que terminou: interface list mostra o seu pacote; service type, o tipo novo
- Antes da entrega do TP1, não mexa nisso: nada desta parte entra nela
- Desafio: quanto pesa informar, em vez de mostrar? ros2 topic bw nos dois tópicos

Como saber que a entrega saiu

```
git status --short                          # nada: está tudo commitado
git tag --list                              # tp1
git ls-remote --tags origin                 # refs/tags/tp1 e tp1^{}   (a tag chegou ao GitHub)
git log --oneline -1 entrega-tp1            # o mesmo commit da linha tp1^{}
```

E, no Moodle, o envio CONCLUÍDO — não em rascunho. Travou e nada na página bate? Infnet.Online, com o comando exato, a saída completa e o passo em que você está. As datas estão na página.

Na próxima: a tarefa que demora, avisa do progresso e pode ser cancelada.

Ações — a terceira forma de comunicação do ROS 2, e a que descreve quase tudo o que um robô faz. Ela entra no pacote de interfaces que você cria na parte 2 da tarefa. Antes disso, uma coisa só: o TP1 entregue.